

# OpenMythos 3B — Complete Step-by-Step Guide for Ubuntu

**Audience:** Complete beginners. No prior experience with AI, Python, or Linux required.

**Hardware:** A machine running Ubuntu (22.04 or 24.04 recommended) with 2 × NVIDIA RTX 3060 GPUs.

This guide walks you through every single step, from a fresh Ubuntu install to a running training job. Follow each step in order and do not skip any.

---

## Table of Contents

1. [Check Your Ubuntu Version](#)
  2. [Install NVIDIA Drivers](#)
  3. [Install CUDA Toolkit](#)
  4. [Verify Both GPUs Are Detected](#)
  5. [Install Python 3.11](#)
  6. [Install Git](#)
  7. [Clone the OpenMythos Repository](#)
  8. [Create a Python Virtual Environment](#)
  9. [Install PyTorch with CUDA Support](#)
  10. [Install OpenMythos and Training Dependencies](#)
  11. [Copy the Training Files Into the Repository](#)
  12. [Run the VRAM Check](#)
  13. [Start Training](#)
  14. [Monitor Training Progress](#)
  15. [Run Inference \(Text Generation\)](#)
  16. [Troubleshooting](#)
- 

## What Is a "Terminal"?

Throughout this guide you will type commands into a **terminal** (also called a "shell" or "command line"). To open one:

- Press `Ctrl + Alt + T` on your keyboard, or
- Search for "Terminal" in your applications menu.

Every command in this guide starts with a `$` sign. **Do not type the `$`** — it just means "type this at the command prompt." Press **Enter** after each command to run it.

---

## Step 1: Check Your Ubuntu Version

Open a terminal and type:

Bash

```
$ lsb_release -a
```

You should see output like:

Plain Text

```
No LSB modules are available.  
Distributor ID: Ubuntu  
Description:    Ubuntu 22.04.3 LTS  
Release:        22.04  
Codename:       jammy
```

This guide works best on **Ubuntu 22.04** or **Ubuntu 24.04**. If you have an older version, consider upgrading before continuing.

---

## Step 2: Install NVIDIA Drivers

Your RTX 3060 GPUs need the correct NVIDIA driver. Ubuntu can install this automatically.

**First, check if a driver is already installed:**

Bash

```
$ nvidia-smi
```

If you see a table showing your two GPUs, drivers are already installed — skip to Step 3.

If you see `command not found`, install the driver now:

Bash

```
$ sudo apt update
$ sudo apt install -y ubuntu-drivers-common
$ sudo ubuntu-drivers autoinstall
```

After the installation finishes, **reboot your machine:**

Bash

```
$ sudo reboot
```

After rebooting, open a new terminal and verify the driver is working:

Bash

```
$ nvidia-smi
```

You should see a table like this (the exact numbers will differ):

Plain Text

```
+-----+
| NVIDIA-SMI 535.104.05    Driver Version: 535.104.05    CUDA Version: 12.2    |
|-----+-----+-----+-----+
| GPU   Name               Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+
|    0  NVIDIA GeForce RTX 3060    Off   | 00000000:01:00.0 Off  |
|    1  NVIDIA GeForce RTX 3060    Off   | 00000000:02:00.0 Off  |
+-----+-----+-----+-----+
```

Both GPUs must appear in this table. If only one shows, check that the second card is properly seated in its PCIe slot.

## Step 3: Install CUDA Toolkit

PyTorch needs the CUDA Toolkit to communicate with the GPUs. We will install CUDA 12.1.

Bash

```
$ wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2204/x86_
$ sudo dpkg -i cuda-keyring_1.1-1_all.deb
$ sudo apt update
$ sudo apt install -y cuda-toolkit-12-1
```

**Note for Ubuntu 24.04 users:** Replace `ubuntu2204` with `ubuntu2404` in the URL above.

After installation, add CUDA to your system's PATH so programs can find it. Run these two commands:

Bash

```
$ echo 'export PATH=/usr/local/cuda-12.1/bin:$PATH' >> ~/.bashrc
$ echo 'export LD_LIBRARY_PATH=/usr/local/cuda-12.1/lib64:$LD_LIBRARY_PATH' >> ~/.bashrc
$ source ~/.bashrc
```

Verify CUDA is installed:

Bash

```
$ nvcc --version
```

You should see something like:

Plain Text

```
nvcc: NVIDIA (R ) Cuda compiler driver
Copyright (c) 2005-2023 NVIDIA Corporation
Built on ...
Cuda compilation tools, release 12.1, V12.1.105
```

## Step 4: Verify Both GPUs Are Detected

Run this command to confirm Python can see both GPUs:

Bash

```
$ python3 -c "import subprocess; subprocess.run(['nvidia-smi', '-L'])"
```

Expected output:

Plain Text

```
GPU 0: NVIDIA GeForce RTX 3060 (UUID: GPU-...)
GPU 1: NVIDIA GeForce RTX 3060 (UUID: GPU-...)
```

Both GPUs must be listed. If only one appears, the second GPU may not be properly installed or may need a driver fix.

## Step 5: Install Python 3.11

Ubuntu 22.04 comes with Python 3.10. We recommend Python 3.11 for best compatibility.

Bash

```
$ sudo apt update
$ sudo apt install -y software-properties-common
$ sudo add-apt-repository ppa:deadsnakes/ppa -y
$ sudo apt update
$ sudo apt install -y python3.11 python3.11-venv python3.11-dev
```

Verify the installation:

Bash

```
$ python3.11 --version
```

Expected output: `Python 3.11.x`

---

## Step 6: Install Git

Git is the tool used to download code from GitHub.

Bash

```
$ sudo apt install -y git
$ git --version
```

Expected output: `git version 2.x.x`

---

## Step 7: Clone the OpenMythos Repository

This downloads the OpenMythos source code to your machine. We will put it in your home directory.

Bash

```
$ cd ~
$ git clone https://github.com/kyegomez/OpenMythos.git
$ cd OpenMythos
```

You are now inside the OpenMythos folder. You can verify this by running:

```
Bash
```

```
$ ls
```

You should see files like `README.md` , `pyproject.toml` , and folders like `open_mythos/` , `training/` , etc.

## Step 8: Create a Python Virtual Environment

A virtual environment keeps all the packages for this project isolated from the rest of your system. This prevents version conflicts.

```
Bash
```

```
$ python3.11 -m venv .venv
```

Now **activate** the virtual environment. You must do this every time you open a new terminal to work on this project:

```
Bash
```

```
$ source .venv/bin/activate
```

Your terminal prompt will change to show `(.venv)` at the beginning, like this:

```
Plain Text
```

```
(.venv) user@machine:~/OpenMythos$
```

This confirms the virtual environment is active. **All following commands assume the virtual environment is active.**

To deactivate it later (when you are done), simply type:

```
Bash
```

```
$ deactivate
```

## Step 9: Install PyTorch with CUDA Support

This is the most important dependency. We install PyTorch **2.5.1** with CUDA 12.1 support. It is critical to pin all three packages ( `torch` , `torchvision` , `torchaudio` ) to the **exact same version** — mixing versions causes the dependency conflict error you may have already seen.

Bash

```
$ pip install --upgrade pip
$ pip install \
    torch==2.5.1+cu121 \
    torchvision==0.20.1+cu121 \
    torchaudio==2.5.1+cu121 \
    --index-url https://download.pytorch.org/whl/cu121
```

If you have CUDA 11.8 instead of 12.x, replace `cu121` with `cu118` in all three lines above.

This download is large (approximately 2–3 GB ). Wait for it to complete.

Verify PyTorch can see your GPUs:

Bash

```
$ python3 -c "import torch; print('PyTorch:', torch.__version__); print('CUDA a
```

Expected output:

Plain Text

```
PyTorch: 2.x.x+cu121
CUDA available: True
GPU count: 2
  GPU 0: NVIDIA GeForce RTX 3060
  GPU 1: NVIDIA GeForce RTX 3060
```

If `CUDA available: False` , go back and check Steps 2 and 3.

## Step 10: Install OpenMythos and Training Dependencies

Bash

```
$ pip install open-mythos
$ pip install datasets transformers loguru tqdm
```

Verify the OpenMythos installation:

Bash

```
$ python3 -c "from open_mythos.variants import mythos_3b; cfg = mythos_3b(); pr
```

Expected output: OpenMythos installed OK. dim = 3072

## Step 11: Download and Extract the Training Files

The zip file was attached to the Manus chat message. You need to **download it first** before you can unzip it.

### How to download the zip file

1. Go back to the Manus chat window.
2. Find the message that contains the file attachment `openmythos_3b_dual_rtx3060.zip`.
3. Click the file name or the download icon next to it.
4. Save it to your **Downloads** folder ( `/home/bounce/Downloads/` ).

### Find your Downloads folder path

Once downloaded, confirm the file is there:

Bash

```
$ ls ~/Downloads/
```

You should see `openmythos_3b_dual_rtx3060.zip` listed. If the file is somewhere else (e.g. the Desktop), find it with:

Bash

```
$ find ~ -name 'openmythos_3b_dual_rtx3060.zip' 2>/dev/null
```

This will print the exact path to the file.

### Extract the zip file

Once you know the path, extract it. For example, if it is in `~/Downloads/` :

Bash

```
$ cd ~
$ unzip ~/Downloads/openmythos_3b_dual_rtx3060.zip -d ~/openmythos_3b_files
```

If `unzip` is not installed, install it first:

Bash

```
$ sudo apt install -y unzip
$ unzip ~/Downloads/openmythos_3b_dual_rtx3060.zip -d ~/openmythos_3b_files
```

## Copy the files into the OpenMythos repository

Bash

```
$ cp ~/openmythos_3b_files/openmythos_3b/training/train_dual_3060.py ~/OpenMythos/
$ cp -r ~/openmythos_3b_files/openmythos_3b/configs ~/OpenMythos/
$ cp -r ~/openmythos_3b_files/openmythos_3b/scripts ~/OpenMythos/
$ cp -r ~/openmythos_3b_files/openmythos_3b/docs ~/OpenMythos/
```

Make the shell scripts executable:

Bash

```
$ chmod +x ~/OpenMythos/scripts/setup_env.sh
```

Go back into the OpenMythos directory and re-activate the virtual environment:

Bash

```
$ cd ~/OpenMythos
$ source .venv/bin/activate
```

Verify the files are in place:

Bash

```
$ ls training/
```

You should see `train_dual_3060.py` listed alongside the original files.

## Step 12: Run the VRAM Check

Before starting training, run this script to confirm the model fits in your GPU memory:

Bash

```
$ python3 scripts/check_vram.py
```

This will:

1. Print the parameter count and fp16 weight size.
2. Move the model to your GPU and run a forward + backward pass.
3. Report the actual VRAM used and whether you have enough headroom.

### What to look for in the output:

- Total parameters should be approximately **3.5 billion**.
- fp16 weight size (total) should be approximately **7.0 GB**.
- fp16 per GPU (FSDP ÷ 2) should be approximately **3.5 GB**.
- ✓ VRAM check PASSED — the model runs without errors.

**Note:** The check script loads the full model on a single GPU for testing.

During actual training, FSDP splits the model across both GPUs, so each GPU only holds half the weights. If the check script OOMs but the fp16 weight size is under 12 GB, training will still work fine.

If you see an OOM error, see the [Troubleshooting](#) section.

## Step 13: Start Training

You are now ready to train the model. The training script uses `torchrun`, which is PyTorch's built-in tool for running code across multiple GPUs.

**Make sure you are in the OpenMythos directory with the virtual environment active:**

Bash

```
$ cd ~/OpenMythos  
$ source .venv/bin/activate
```

### Launch training across both GPUs:

Bash

```
$ torchrun --nproc_per_node=2 training/train_dual_3060.py
```

- `--nproc_per_node=2` tells PyTorch to use 2 GPUs on this machine.

You will see output like this after a short startup period:

Plain Text

```
2026-05-22 10:00:00 | INFO | Starting OpenMythos 3B training | world_size=2 | s
2026-05-22 10:00:15 | INFO | Tokenizer vocab_size=200019
2026-05-22 10:00:30 | INFO | Model parameters: 3,012,345,678 (3.01B)
2026-05-22 10:01:00 | INFO | step=      10 | loss=10.8432 | lr=1.50e-06 | tok/s=
2026-05-22 10:01:48 | INFO | step=      20 | loss=10.6201 | lr=3.00e-06 | tok/s=
...
```

The loss value should gradually decrease over time. This is the model learning.

**The first run will download the FineWeb-Edu dataset**, which is approximately 30–50 GB for the `sample-10BT` subset. This will take time depending on your internet connection.

**To stop training at any time**, press `Ctrl + C`. The script saves a checkpoint every 1000 steps, so you will not lose progress.

**To resume training** after stopping, simply run the same `torchrun` command again. It will automatically detect the latest checkpoint and continue from where it left off.

## Step 14: Monitor Training Progress

### Check GPU Usage

Open a **second terminal** (leave the training running in the first one) and run:

Bash

```
$ watch -n 2 nvidia-smi
```

This refreshes the GPU status every 2 seconds. You should see:

- Both GPUs at high utilization (60–100%).
- Memory usage around 10–11 GB per GPU.

Press `Ctrl + C` to stop watching.

### Check Disk Space

Checkpoints are saved to `~/OpenMythos/checkpoints/3b_dual_3060/`. Each checkpoint is approximately 6–12 GB. The script keeps only the 3 most recent ones. Make sure you have at least 50 GB of free disk space:

Bash

```
$ df -h ~
```

## Check Training Logs

The training script prints logs to the terminal. Key things to watch:

| Field              | What It Means                      | What's Good                                               |
|--------------------|------------------------------------|-----------------------------------------------------------|
| <code>loss</code>  | How well the model is learning     | Should decrease over time                                 |
| <code>lr</code>    | Current learning rate              | Rises during warmup, then decays                          |
| <code>tok/s</code> | Training speed (tokens per second) | Higher is better; ~10,000–15,000 is typical for dual 3060 |

## Step 15: Run Inference (Text Generation)

Once training has run for a while (at least a few thousand steps), you can generate text with the model.

**With a trained checkpoint:**

Bash

```
$ python3 scripts/inference.py \  
  --checkpoint checkpoints/3b_dual_3060/step_0001000.pt \  
  --prompt "The most important thing about artificial intelligence is" \  
  --max_new_tokens 100 \  
  --loops 8
```

Replace `step_0001000.pt` with the name of your latest checkpoint file. To see what checkpoints you have:

Bash

```
$ ls checkpoints/3b_dual_3060/
```

### Without a checkpoint (architecture test only):

Bash

```
$ python3 scripts/inference.py --prompt "Hello, world!"
```

This runs with random weights, so the output will be nonsense — but it confirms the inference pipeline works.

### Inference options:

| Option                        | Default | Description                                                                     |
|-------------------------------|---------|---------------------------------------------------------------------------------|
| <code>--loops</code>          | 8       | Number of recurrent loops. More loops = deeper reasoning but slower.            |
| <code>--max_new_tokens</code> | 128     | How many tokens to generate.                                                    |
| <code>--temperature</code>    | 0.8     | Randomness. Lower (e.g. 0.3) = more focused. Higher (e.g. 1.2) = more creative. |
| <code>--top_k</code>          | 50      | Only sample from the top 50 most likely next tokens.                            |

## Troubleshooting

### "CUDA out of memory" during training

This means the GPU does not have enough VRAM. Try these fixes in order:

**Fix 1: Reduce sequence length.** Open `training/train_dual_3060.py` in a text editor and change:

Python

```
SEQ_LEN: int = 2048
```

to:

Python

```
SEQ_LEN: int = 1024
```

Save the file and restart training.

**Fix 2: Enable CPU offloading.** In the same file, change:

```
Python
```

```
CPU_OFFLOAD: bool = False
```

to:

```
Python
```

```
CPU_OFFLOAD: bool = True
```

This moves optimizer states to system RAM. Training will be slower ( $3-5\times$ ) but will use less VRAM.

**Fix 3: Reduce training loops.** Change:

```
Python
```

```
TRAIN_N_LOOPS: int = 8
```

to:

```
Python
```

```
TRAIN_N_LOOPS: int = 4
```

**Fix 4: Kill other GPU processes.** Run `nvidia-smi` and check if any other program is using GPU memory. Close any games, other AI applications, or browser windows with GPU acceleration.

## "command not found: torchrun"

The virtual environment is not active. Run:

```
Bash
```

```
$ source ~/OpenMythos/.venv/bin/activate
```

## "ModuleNotFoundError: No module named 'open\_mythos'"

The package is not installed in the active environment. Run:

Bash

```
$ pip install open-mythos
```

## Training is very slow (less than 5,000 tokens/second)

This is often caused by the dataset download happening in the background. Wait for the first few batches to complete — speed should stabilize after the initial data loading.

If it remains slow, check that both GPUs are being used:

Bash

```
$ nvidia-smi
```

Both should show high GPU utilization. If only one shows activity, the distributed launch may have failed. Try stopping and restarting with `torchrun --nproc_per_node=2 training/train_dual_3060.py`.

## CUBLAS\_STATUS\_ALLOC\_FAILED or OOM when running check\_vram.py

If you see:

Plain Text

```
RuntimeError: CUDA error: CUBLAS_STATUS_ALLOC_FAILED
```

or the VRAM check shows `VRAM after model load: 12.175 GB`, this means the model is too large for a single GPU. This is caused by using the **upstream mythos\_3b()** **config**, which is actually ~12.76 billion parameters (not 3B) because of its large MoE settings ( `n_experts=64, expert_dim=4096` ).

The training files in this package use a **corrected config** ( `n_experts=32, expert_dim=2048` ) that gives a genuine ~3.51B parameter model (~7 GB fp16).

If you are seeing this error, make sure you copied the files from the zip archive correctly in Step 11. Specifically, verify that `configs/mythos_3b_rtx3060.py` exists in your OpenMythos folder:

Bash

```
$ ls ~/OpenMythos/configs/  
# Should show: mythos_3b_rtx3060.py
```

If the file is missing, re-run the copy commands from Step 11.

## pip dependency conflict: torchaudio/torchvision incompatible with torch

If you see an error like:

Plain Text

```
torchaudio X.X requires torch==2.5.1, but you have torch 2.11.0 which is incompati
```

This means `torch`, `torchvision`, and `torchaudio` were installed at different versions. Fix it by uninstalling all three and reinstalling them together with matching pinned versions:

Bash

```
$ pip uninstall -y torch torchvision torchaudio  
$ pip install \  
    torch==2.5.1+cu121 \  
    torchvision==0.20.1+cu121 \  
    torchaudio==2.5.1+cu121 \  
    --index-url https://download.pytorch.org/whl/cu121
```

For CUDA 11.8, replace `cu121` with `cu118` in all three lines. After reinstalling, verify with:

Bash

```
$ python3 -c "import torch; print(torch.__version__)"
```

You should see `2.5.1+cu121`.

## The loss is not decreasing

A loss that stays flat or increases after the warmup period (2000 steps) indicates a problem. Check:

1. The learning rate is not too high. Try reducing `MAX_LR` from `3e-4` to `1e-4`.
2. Gradient clipping is on (`GRAD_CLIP: float = 1.0` should be set).
3. The dataset is loading correctly — check for error messages in the terminal output.

# How to edit the training configuration

All training settings are at the top of `training/train_dual_3060.py` . Open it with a text editor:

```
Bash

$ nano training/train_dual_3060.py
```

Use the arrow keys to navigate. Press `Ctrl + O` then `Enter` to save. Press `Ctrl + X` to exit.

The key settings are:

```
Python

CPU_OFFLOAD: bool = False           # Set True if running out of VRAM
MICRO_BATCH_SIZE: int = 1           # Batch size per GPU (keep at 1)
GRADIENT_ACCUM: int = 16            # Steps before each optimizer update
SEQ_LEN: int = 2048                  # Sequence length (reduce if OOM)
MAX_LR: float = 3e-4                 # Peak learning rate
TOTAL_STEPS: int = 100_000           # Total training steps
TRAIN_N_LOOPS: int = 8               # Recurrent loops during training
DATASET_SUBSET: str = "sample-10BT" # Dataset size
```

## Quick Reference: Commands You Will Use Most

| Task                         | Command                                                                            |
|------------------------------|------------------------------------------------------------------------------------|
| Activate virtual environment | <code>source ~/OpenMythos/.venv/bin/activate</code>                                |
| Check GPU status             | <code>nvidia-smi</code>                                                            |
| Run VRAM check               | <code>python3 scripts/check_vram.py</code>                                         |
| Start training (2 GPUs)      | <code>torchrun --nproc_per_node=2 training/train_dual_3060.py</code>               |
| Monitor GPU live             | <code>watch -n 2 nvidia-smi</code>                                                 |
| List checkpoints             | <code>ls checkpoints/3b_dual_3060/</code>                                          |
| Run inference                | <code>python3 scripts/inference.py --checkpoint &lt;path&gt; --prompt "..."</code> |
| Check disk space             | <code>df -h ~</code>                                                               |

## What Happens During Training?

The model is learning to predict the next word in a sentence by reading billions of words from the FineWeb-Edu dataset (a large collection of educational web text). Each "step" is one update to the model's weights based on a batch of text. After enough steps, the model will have learned enough about language to generate coherent text.

The **loss** is a measure of how wrong the model's predictions are. A loss of 10 means the model is essentially guessing randomly. A loss of 3–4 means the model has learned meaningful patterns. A well-trained language model typically reaches a loss of 2–3 on this type of dataset.

With a dual RTX 3060 setup, training for 100,000 steps on the `sample-10BT` subset will take approximately **3–7 days** depending on your exact hardware configuration and system load. You do not need to keep the training running continuously — you can stop and resume at any time.